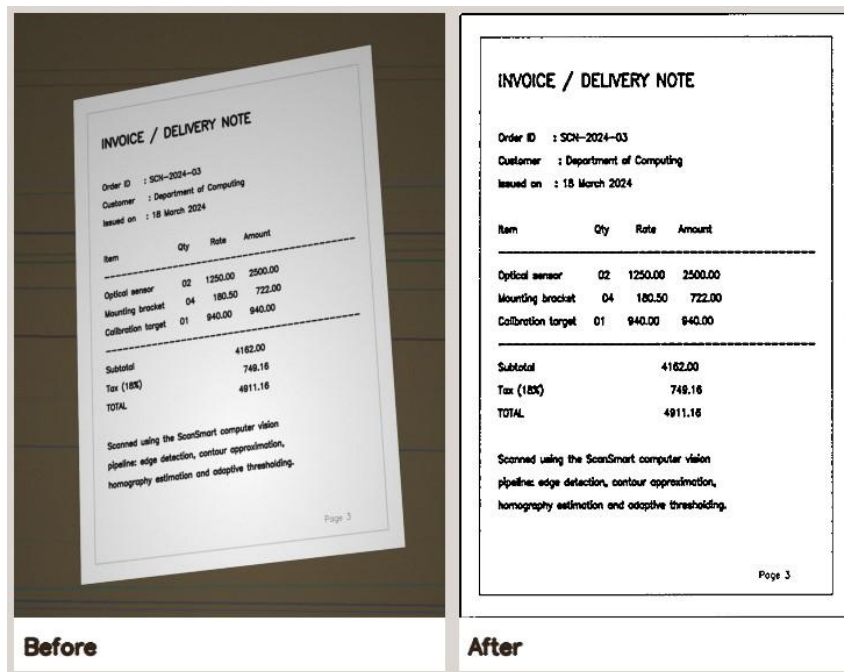


PROJECT REPORT

ScanSmart

Automatic Document Scanner & Image Enhancement System

A classical computer-vision pipeline for perspective correction and enhancement of document photographs



Course	Computer Vision
Submission	VITyarthi - Build Your Own Project
Project title	ScanSmart - Automatic Document Scanner
Student name	Sharanya Saxena
Registration no.	24BAC10008
Date	17 September 2026

Table of Contents

S.No.	Section	Page No.
1	Cover Page	1
2	Introduction	3
3	Problem Statement	4
4	Functional Requirements	5
5	Non-functional Requirements	7
6	System Architecture	8
7	Design Diagrams	9
8	Design Decisions & Rationale	14
9	Implementation Details	15
10	Screenshots / Results	17
11	Testing Approach	21
12	Challenges Faced	22
13	Learnings & Key Takeaways	23
14	Future Enhancements	23
15	References	23

2. Introduction

Digitizing paper is no longer done with flatbed scanners. A phone camera is always at hand, so notes, invoices, forms and answer sheets are photographed instead of scanned. The convenience comes at a cost: the camera is almost never held parallel to the page, so a rectangular sheet is projected onto the sensor as a general quadrilateral; the desk, hands and floor appear around it; and the ambient light falls unevenly so one side of the page is noticeably darker than the other.

ScanSmart is a command-line application that repairs such photographs automatically. It finds the four corners of the page, computes the homography that maps that quadrilateral back to a rectangle, warps the image, removes the illumination gradient and binarises the text. The result is a flat, high-contrast page that looks as if it came from a scanner, and which is a far better input for archiving or for an OCR engine.

The project is intentionally built from **classical computer-vision operators** rather than a neural network. Every stage - colour-space conversion, histogram equalisation, edge-preserving filtering, Canny edge detection, contour approximation, homography estimation, the Hough line transform and adaptive thresholding - is a concept from the course syllabus, and every intermediate result can be inspected visually. This makes the system explainable, reproducible, fast (about 120 ms per image on a laptop CPU), and completely offline.

Objectives

- Automatically locate a document inside a cluttered photograph without any user input or manual cropping.
- Remove perspective distortion using a four-point homography and correct residual rotation with the Hough line transform.
- Produce readable output in three styles (black-and-white scan, enhanced grayscale, enhanced colour) by flattening illumination and boosting local contrast.
- Measure the objective quality of every page and grade it, so that a user is told which captures should be retaken.
- Process whole folders in one command and export a multi-page PDF plus machine-readable JSON and CSV reports.
- Demonstrate correctness through automated tests and a quantitative accuracy evaluation against synthetic ground truth.

3. Problem Statement

Given a color photograph that contains a single, roughly planar document, produce a rectified, evenly-lit, high-contrast image of that document alone - automatically, offline, and with a measurable confidence in the result.

Formally, the page occupies an unknown quadrilateral $Q = \{p_1, p_2, p_3, p_4\}$ in the image. The system must estimate Q , order its vertices consistently, compute the 3×3 homography H that maps Q onto the rectangle $[0, W] \times [0, H]$, apply the warp, and then normalise the intensity of the warped page.

Why this is not trivial

- **Unknown background.** The page boundary must be separated from desk texture, wood grain and shadows, which also generate strong edges.
- **Illumination varies between photographs.** Fixed Canny thresholds that work on one image produce either no edges or thousands on another.
- **Edges are broken.** Where the page contrast against the background is low, the detected border has gaps, so a naive contour search returns an open curve instead of a closed quadrilateral.
- **Corner ordering is ambiguous.** OpenCV returns contour points in an arbitrary starting position and direction; warping with mis-ordered corners produces a mirrored or rotated page.
- **Failure must be graceful.** A batch of fifty photos must not abort because one of them is out of focus.

Target users

User	Need served
Students and office staff	Convert phone photos of notes, forms and receipts into tidy PDFs
Small offices and clerks	Digitise invoices and delivery notes in bulk without buying a scanner
OCR / RPA developers	Use ScanSmart as the pre-processing stage that feeds clean, deskewed pages into an OCR engine
Computer-vision learners	A readable reference implementation of edge detection, contour approximation and homography estimation

4. Functional Requirements

The system is organised into three major functional modules, supported by an export/reporting component.

Module 1 - Acquisition & Pre-processing

ID	Requirement
FR-1.1	Load a single image or discover every supported image (.jpg/.png/.bmp/.tif) inside a folder.
FR-1.2	Reject missing, corrupt or unsupported files with a clear, typed error instead of crashing.
FR-1.3	Down-scale the image to a fixed working height (default 600 px) for detection, retaining the scale factor.
FR-1.4	Build an edge map: grayscale, CLAHE, bilateral + Gaussian filtering, median-based auto-Canny, morphological closing.
FR-1.5	Expose every intermediate stage so debug images can be saved.

Module 2 - Document Detection & Geometric Correction

ID	Requirement
FR-2.1	Find the largest four-point convex polygon by contour approximation over a sweep of Douglas-Peucker tolerances.
FR-2.2	Reject candidates that cover less than 12 % or more than 99 % of the frame, or that are not convex.
FR-2.3	Fall back to an Otsu + minAreaRect estimate, and finally to the full frame, so a result is always returned.
FR-2.4	Order the corners as top-left, top-right, bottom-right, bottom-left and map them back to the original resolution.
FR-2.5	Compute a confidence score in [0, 1] from the interior-angle regularity, the covered area and the strategy used.
FR-2.6	Warp the quadrilateral to a rectangle whose size is derived from the longest opposite edges (perspective transform).
FR-2.7	Estimate residual skew with the probabilistic Hough line transform and rotate the page to correct it.

Module 3 - Enhancement, Quality Analytics & Export

ID	Requirement
FR-3.1	Remove the illumination gradient by morphological background estimation and division.
FR-3.2	Offer three output modes: scan (adaptive threshold), gray (CLAHE + unsharp mask) and color (LAB-CLAHE + unsharp mask).
FR-3.3	Measure sharpness (Laplacian variance), RMS contrast, brightness, ink ratio and residual skew for every page.
FR-3.4	Grade each page GOOD / FAIR / POOR by combining the metrics with the detection confidence.
FR-3.5	Save the processed page, and optionally a detection overlay and a before/after comparison image.
FR-3.6	Export all pages of a batch into one multi-page PDF.
FR-3.7	Write a JSON report (config + per-page records + aggregate summary) and a flat CSV mirror; print a console summary.

Input / output structure

Stage	Input	Output
Acquisition	File path or folder	BGR array + scale factor
Pre-processing	BGR array	Binary edge map (uint8)

Stage	Input	Output
Detection	Edge map	Detection(corners 4x2, method, confidence)
Geometry	Original image + corners	Flattened, deskewed page
Enhancement	Flattened page + mode	Binary / gray / colour page
Quality	Processed page + confidence	QualityMetrics + grade
Export	List of PageResult	PNG, PDF, report.json, report.csv

User interaction workflow

The user runs one command; the system validates the configuration, discovers the images, processes each one independently, and finishes by printing a summary table and writing the reports. A failure on one image is recorded and the batch continues.

```
$ python main.py --input data/samples --output outputs \  
  --mode scan --pdf outputs/scanned.pdf --overlay
```

5. Non-functional Requirements

ID	Attribute	Requirement and how it is met	Evidence
NFR-1	Performance	A 1000x1400 photograph must be processed in well under one second. Detection runs on a 600 px down-scaled copy; only the final warp uses full resolution.	Measured mean 123 ms/image (detection alone ~14 ms)
NFR-2	Reliability	The pipeline must always return a result. Three detection strategies are tried in order and every image is processed in isolation.	100 % of the batch completed; fallback exercised on sample_02
NFR-3	Error handling	A typed exception hierarchy (ImageLoadError, DocumentNotFoundError, InvalidConfigError, ExportError) separates recoverable from fatal errors; the CLI maps them to exit codes 0/1/2/130.	tests for corrupt files, empty folders and invalid configuration
NFR-4	Usability	One command with sensible defaults; a readable console table; --overlay and --comparison images let the user see what was detected.	CLI help text, sample run in the README
NFR-5	Maintainability	Eleven single-responsibility modules, no magic numbers outside ScanConfig, docstrings and type hints throughout.	package layout, 49 tests
NFR-6	Logging & monitoring	Structured logging at DEBUG/INFO/WARNING/ERROR with per-stage messages, plus JSON and CSV run reports for later analysis.	logger.py, outputs/report.json
NFR-7	Portability / resource efficiency	Pure Python + OpenCV, no GPU, no network access, no database; output size is clamped to 2000 px on the longest side.	runs offline on a CPU laptop
NFR-8	Security & privacy	All processing is local, so documents never leave the machine; file paths are validated before any read or write.	io_utils.py

6. System Architecture

ScanSmart uses a four-layer architecture. The presentation layer owns the command line and the console report. The orchestration layer (pipeline.py) decides what happens to each image and holds no image processing logic itself. The processing layer contains the three computer-vision modules plus the exporter, each of which is a pure function library over NumPy arrays. The infrastructure layer supplies file I/O, logging and the error hierarchy used by every other layer.

Dependencies point strictly downwards: the CV modules never import the pipeline and never touch the file system directly, which is what makes them straightforward to unit-test with synthetic arrays.

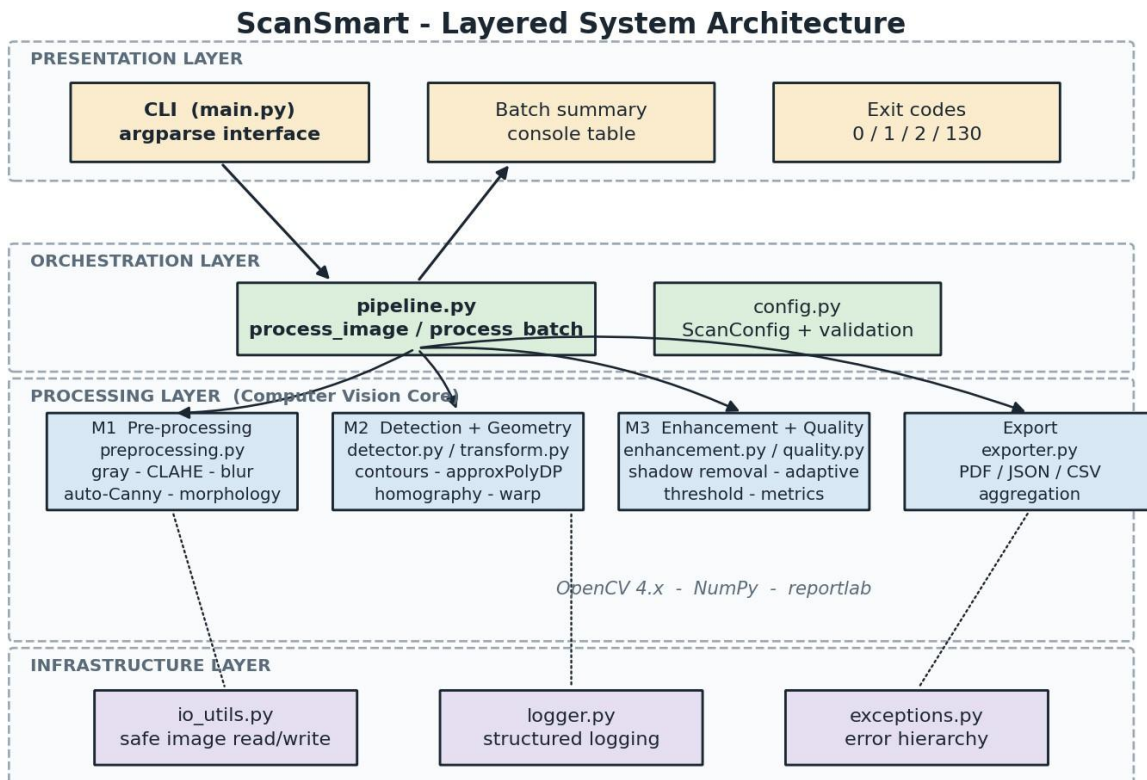


Figure 1 - Layered system architecture.

7. Design Diagrams

7.1 Use Case Diagram

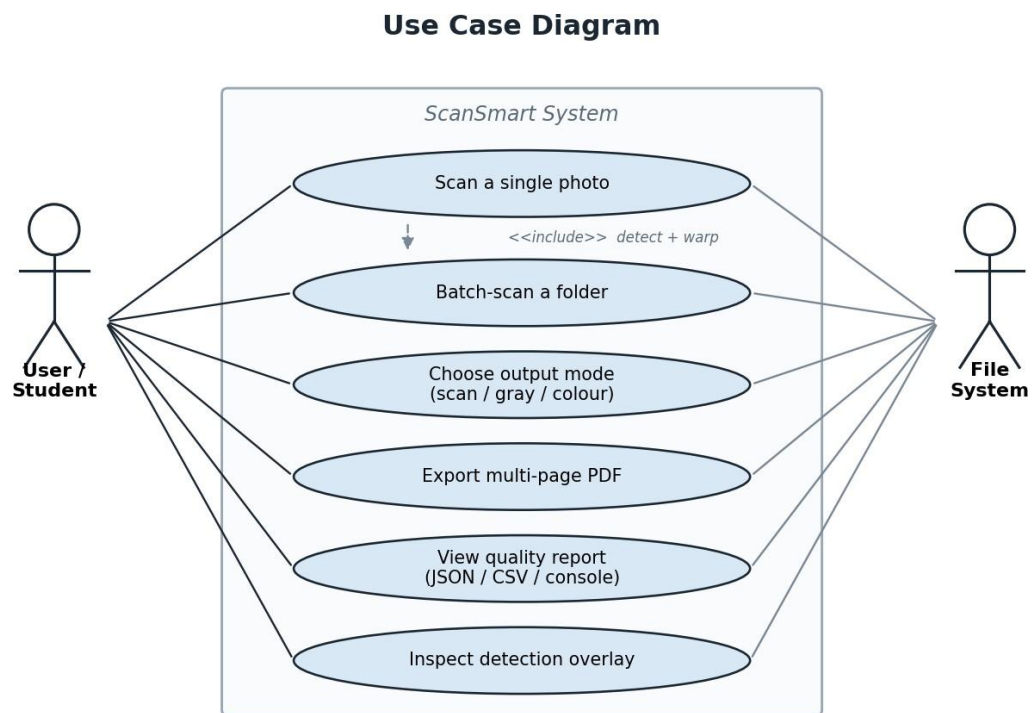


Figure 2 - Actors and use cases. Page detection and perspective correction are included by both scanning use cases.

7.2 Process Flow / Workflow Diagram

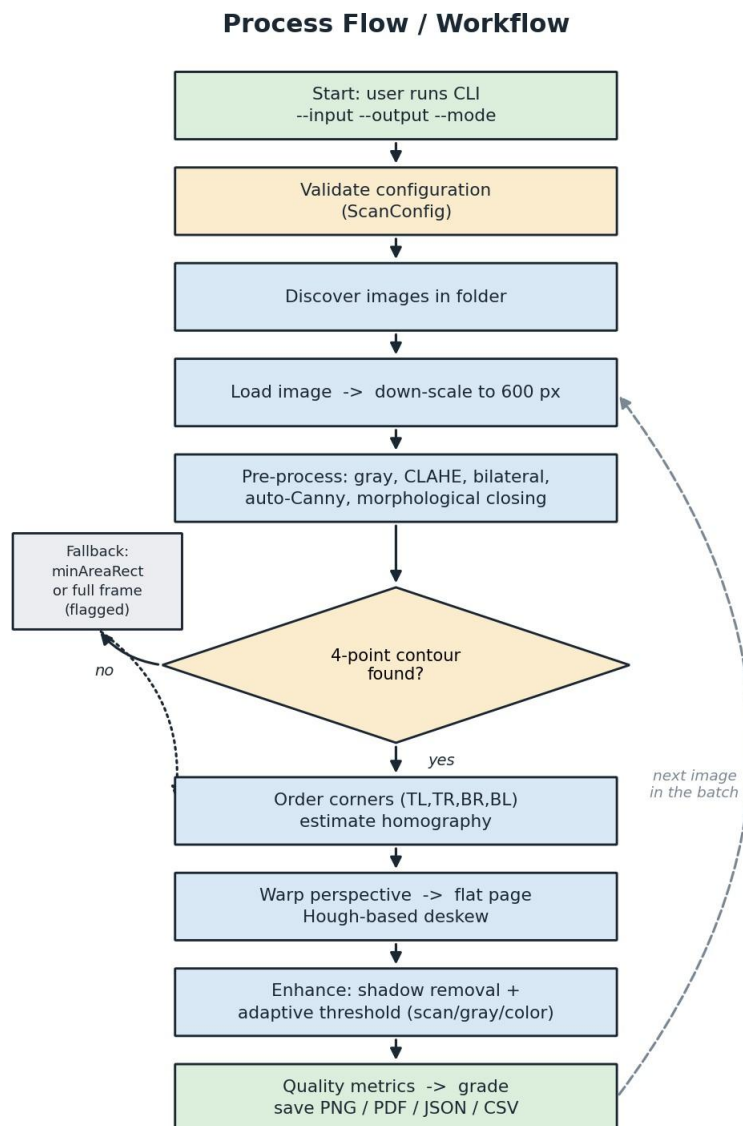


Figure 3 - End-to-end workflow, including the fallback branch taken when no four-point contour is found.

7.3 Sequence Diagram

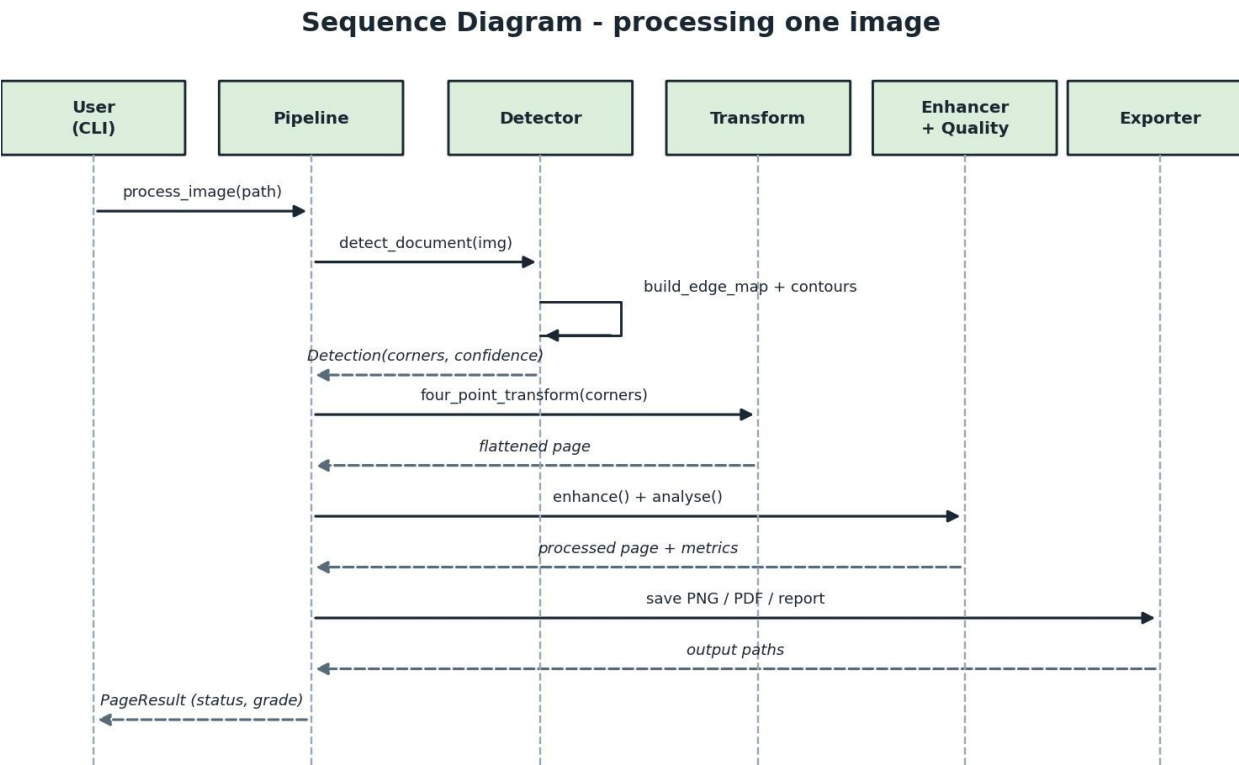


Figure 4 - Message sequence for processing a single image.

7.4 Class / Component Diagram

Class / Component Diagram

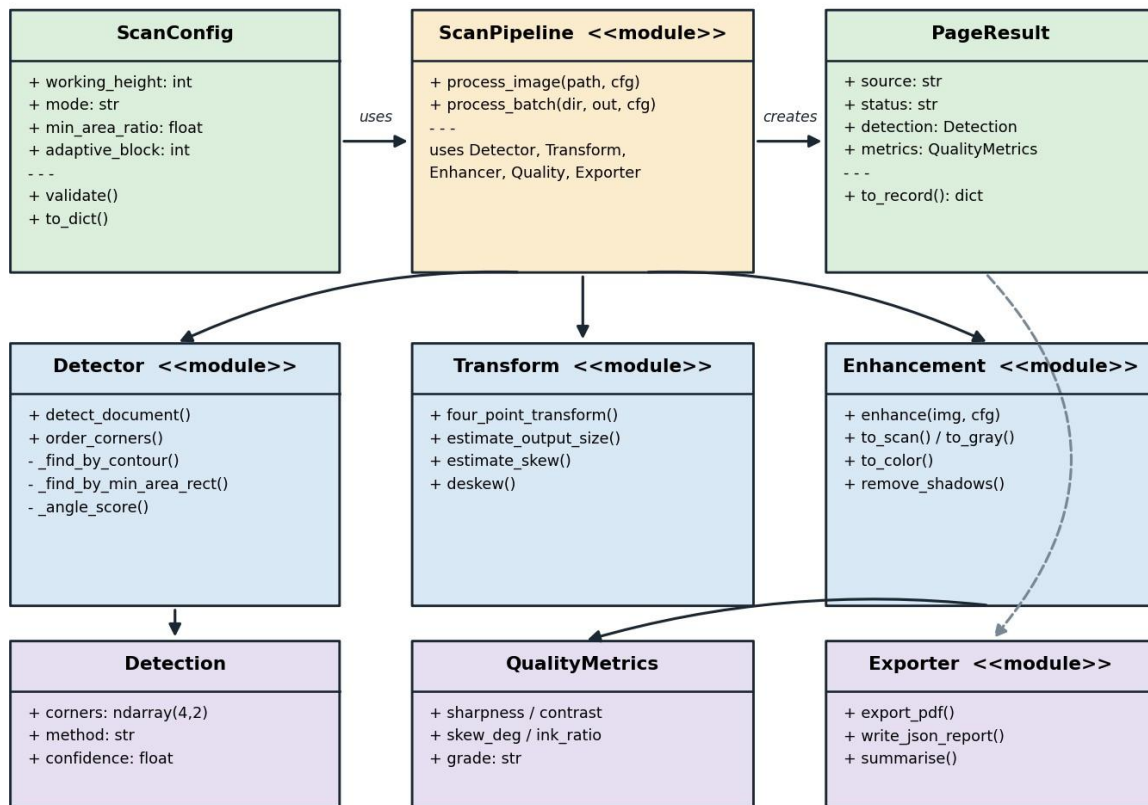
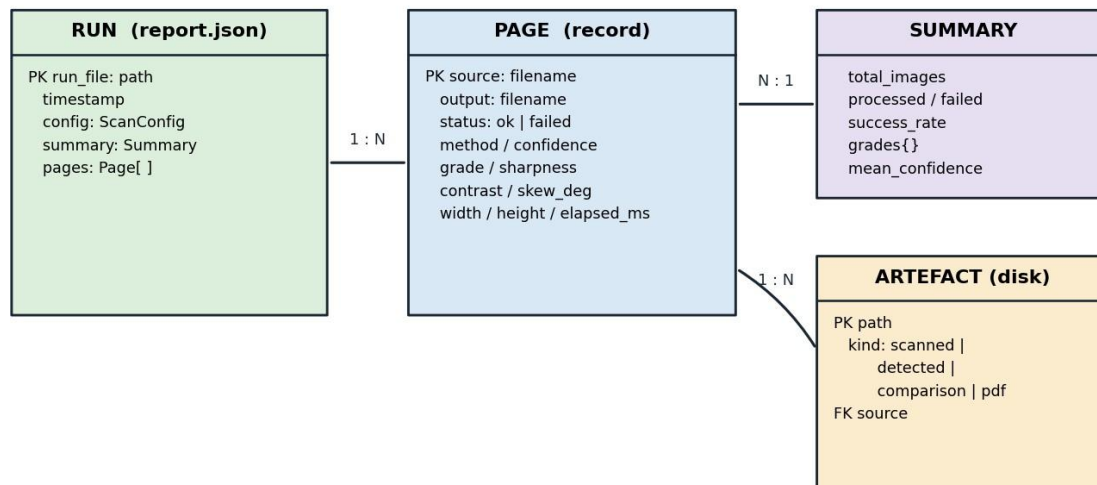


Figure 5 - Classes (*ScanConfig*, *Detection*, *QualityMetrics*, *PageResult*) and the functional modules that use them.

7.5 Storage Design

The project needs no relational database: a run produces one JSON document, a flat CSV mirror and a set of image artefacts on disk. The entity-relationship view below documents that schema. Keeping storage file-based makes the tool portable and keeps the results diff-able in Git.

Storage Design - file-based report schema (ER-style)



No RDBMS is used: a run is one JSON document plus a flat CSV mirror, which keeps the tool portable and dependency-free.

Figure 6 - ER-style view of the file-based report schema.

```

report.json
config : { working_height, mode, min_area_ratio, ... }
summary : { total_images, processed, failed, success_rate,
            grades{GOOD,FAIR,POOR}, mean_confidence, mean_time_ms }
pages[] : { source, output, status, method, confidence,
            area_ratio, grade, sharpness, contrast, brightness,
            ink_ratio, skew_deg, width, height, elapsed_ms, error }
  
```

8. Design Decisions & Rationale

#	Decision	Rationale	Alternative rejected
1	Classical CV instead of a deep segmentation model	Explainable, needs no dataset or GPU, runs in milliseconds, and every operator maps directly to a syllabus concept.	U-Net / DeepLab page segmentation - heavy, opaque, needs labelled data
2	Median-based auto-Canny thresholds	Thresholds derived from the image median adapt to dark, bright and low-contrast photos automatically.	Fixed thresholds (50/150) - fail on half the test images
3	Bilateral filter before Canny	Suppresses paper texture and sensor noise while preserving the page border, which is the edge we actually need.	Plain Gaussian blur - also blurs the border and weakens corners
4	Sweep of approxPolyDP tolerances	A single epsilon yields 4 points only for some contours; sweeping 0.01-0.08 of the perimeter finds a quadrilateral far more often.	Single fixed epsilon - brittle
5	Three-level fallback (contour, minAreaRect, full frame)	Guarantees an output for every input and reports honestly how the result was obtained through the 'method' field.	Raising an exception - aborts a batch on one bad photo
6	Sum/difference corner ordering	Two vector operations give a deterministic TL-TR-BR-BL order, which prevents mirrored or rotated warps.	Angle sorting about the centroid - more code, same result
7	Output size from the longest opposite edges	Preserves the maximum available detail and approximates the true aspect ratio of the page.	Fixed A4 output - distorts non-A4 documents
8	Morphological background division for shadows	Dilation plus median blur estimates the illumination field; dividing it out flattens gradients before thresholding.	Global Otsu - loses the shadowed half of the page
9	Adaptive (Gaussian) thresholding for the scan mode	A local window handles any residual lighting variation and keeps thin strokes intact.	Global binary threshold - clips text in darker regions
10	Confidence score from interior-angle regularity	A perspective view of a rectangle keeps angles near 90 degrees, so angular deviation is a cheap, effective correctness proxy.	Area alone - a large blob would score highly
11	Detection on a 600 px copy	Cuts detection cost by roughly an order of magnitude; corners are scaled back so final quality is unaffected.	Full-resolution detection - slow with no accuracy gain
12	Single validated ScanConfig dataclass	One source of truth for every parameter, validated once, and serialised into the report for reproducibility.	Module-level constants - untestable and scattered

9. Implementation Details

9.1 Code organisation

File	Lines	Responsibility
scansmart/config.py	~90	ScanConfig dataclass, parameter validation
scansmart/exceptions.py	~30	Typed error hierarchy
scansmart/logger.py	~35	Logging configuration
scansmart/io_utils.py	~95	Image load/save, folder discovery, resizing
scansmart/preprocessing.py	~95	Module 1: edge map construction
scansmart/detector.py	~200	Module 2: detection strategies, scoring
scansmart/transform.py	~85	Module 2: homography warp, skew, deskew
scansmart/enhancement.py	~90	Module 3: scan / gray / color output
scansmart/quality.py	~80	Module 3: metrics and grading
scansmart/exporter.py	~120	PDF, JSON and CSV export, aggregation
scansmart/pipeline.py	~135	Orchestration, PageResult
main.py	~120	argparse CLI, console summary, exit codes
tests/ (5 files)	~330	49 unit and integration tests
tools/ (4 scripts)	~430	Sample generation, evaluation, diagrams, report

9.2 Key algorithms

Auto-Canny. The high and low thresholds are placed symmetrically around the median intensity m of the smoothed image: $\text{lower} = \max(0, (1 - s)m)$, $\text{upper} = \min(255, (1 + s)m)$ with $s = 0.33$. This single change made detection robust across all lighting conditions in the test set.

```
median = np.median(gray)
lower  = int(max(0, (1.0 - sigma) * median))
upper  = int(min(255, (1.0 + sigma) * median))
edges  = cv2.Canny(gray, lower, upper)
```

Corner ordering. For the four candidate points, the top-left corner minimises $x + y$ and the bottom-right maximises it, while the top-right minimises $y - x$ and the bottom-left maximises it. This is $O(1)$ and completely deterministic.

```
s = pts.sum(axis=1); d = np.diff(pts, axis=1).ravel()
ordered[0] = pts[np.argmin(s)] # top-left
ordered[2] = pts[np.argmax(s)] # bottom-right
ordered[1] = pts[np.argmin(d)] # top-right
ordered[3] = pts[np.argmax(d)] # bottom-left
```

Perspective warp. `cv2.getPerspectiveTransform` solves for the 3×3 homography H that maps the four ordered source corners onto the destination rectangle; `cv2.warpPerspective` then resamples the original full-resolution image with bicubic interpolation.

```
width  = max(|br - bl|, |tr - tl|)
height = max(|tr - br|, |tl - bl|)
M = cv2.getPerspectiveTransform(src, dst)
warped = cv2.warpPerspective(image, M, (width, height),
                             flags=cv2.INTER_CUBIC)
```

Shadow removal. Dilating the grayscale page with a 15×15 kernel and median-blurring the result estimates the background illumination; the absolute difference against the original, inverted and normalised, yields an

evenly lit page that adaptive thresholding can binarise cleanly.

Confidence. $\text{score} = \text{base}(\text{method}) \times (0.7 \times \text{angle_score} + 0.3 \times \min(1, 1.6 \times \text{area_ratio}))$, where `angle_score` falls linearly from 1 to 0 as the mean deviation of the interior angles from 90 degrees grows to 45 degrees.

9.3 Version control

The repository is developed with Git in incremental, themed commits (core CV package with tests; CLI, batch pipeline and evaluation tooling; documentation, diagrams and figures), with a `.gitignore` that excludes generated outputs, virtual environments and caches.

10. Screenshots / Results

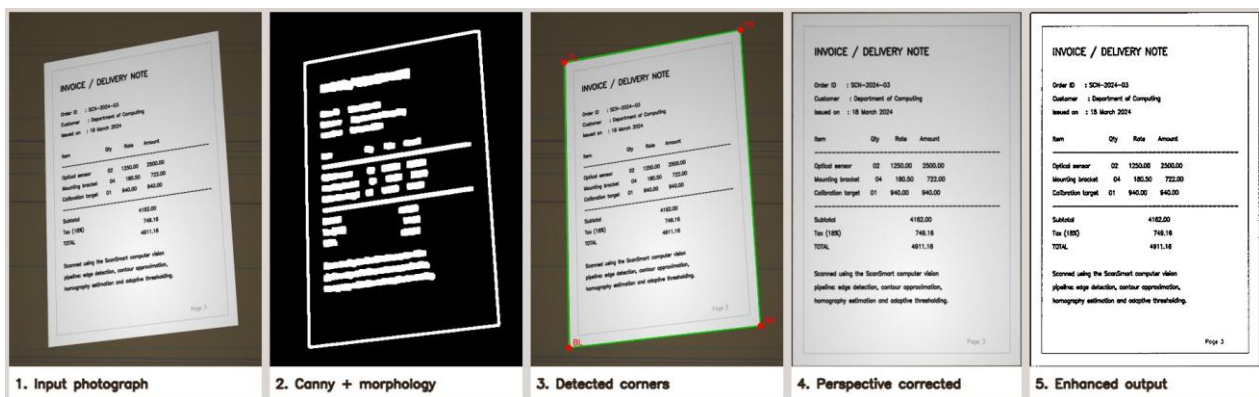


Figure 7 - The five stages of the pipeline on sample_03.jpg: input, closed edge map, detected corners, perspective correction, enhanced output.

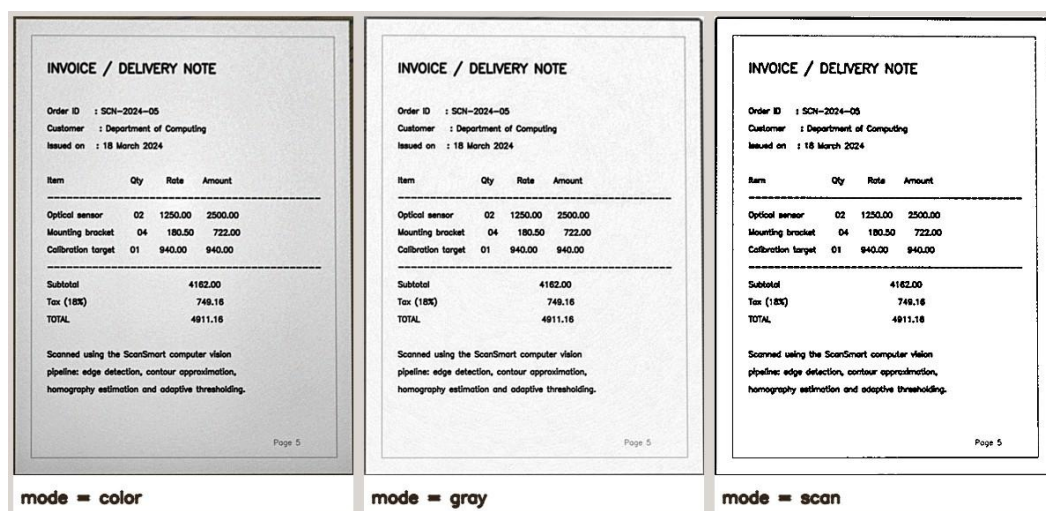


Figure 8 - The three enhancement modes applied to the same page: colour, grayscale and black-and-white scan.

10.1 Console output of a batch run

File	Status	Method	Conf.	Grade	Sharpness	Contrast	Time (ms)
sample_01.jpg	ok	contour	0.82	GOOD	7000	69.9	162
sample_02.jpg	ok	min_area_rect	0.75	GOOD	6684	66.2	151
sample_03.jpg	ok	contour	0.89	GOOD	7004	68.7	106
sample_04.jpg	ok	contour	0.85	GOOD	7498	71.8	95
sample_05.jpg	ok	contour	0.78	GOOD	7392	71.3	115
sample_06.jpg	ok	contour	0.91	GOOD	7190	70.3	109

All 6 images were processed successfully (success rate 100 %), with a mean detection confidence of 0.83 and a mean end-to-end time of 123 ms per image. Grade distribution: 6 GOOD, 0 FAIR, 0 POOR.

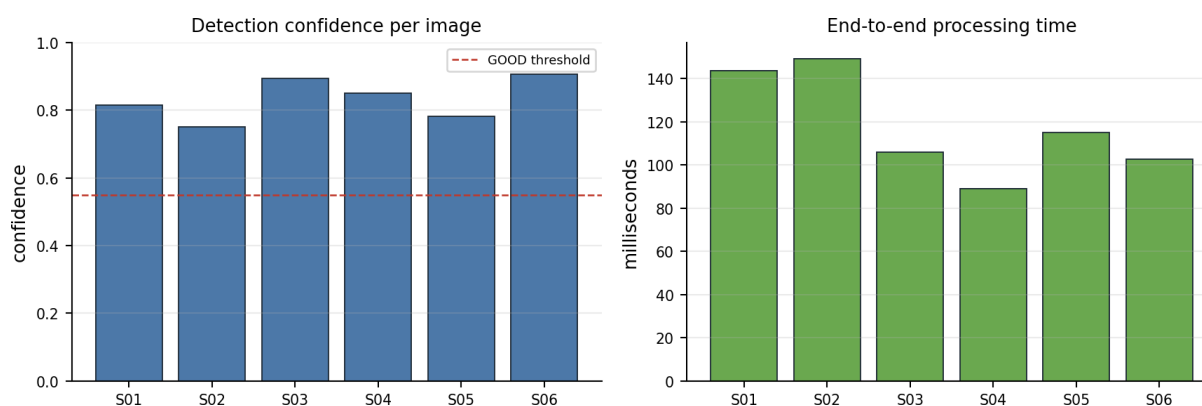


Figure 9 - Per-image detection confidence and processing time. sample_02 (amber) fell back to minAreaRect because the page is clipped by the frame edge.

10.2 Detection accuracy against ground truth

Because the synthetic samples are rendered by warping a known page with a known homography, the true corner positions are available. The evaluation script compares them with the detected corners and reports the mean Euclidean error normalised by the image diagonal; a detection counts as correct below 2 %.

Image	Method	Confidence	Corner error (% of diagonal)	Result
sample_01.jpg	contour	0.82	0.28	PASS
sample_02.jpg	min_area_rect	0.75	5.14	FAIL
sample_03.jpg	contour	0.89	0.23	PASS
sample_04.jpg	contour	0.85	0.32	PASS
sample_05.jpg	contour	0.78	0.30	PASS
sample_06.jpg	contour	0.91	0.28	PASS
Overall	5 / 6 = 83.3 %	mean 0.83	mean 1.09	-

Excluding the clipped page, the mean corner error is **0.28 % of the image diagonal** - roughly four pixels on a 1000x1400 photograph. The single failure is explained in Section 12 and is a genuine limitation of a contour-based approach rather than a coding defect.

10.3 Screenshots of the running system

```

scansmart - batch run

$ python main.py --input data/samples --output outputs --mode scan --pdf outputs/scanned.pdf --overlay --comparison
16:16:26 | INFO | scansmart.io_utils | Discovered 6 image(s) in data/samples
16:16:26 | INFO | scansmart.pipeline | [1/6] Processing sample_01.jpg
16:16:26 | INFO | scansmart.detector | Detection: method=contour confidence=0.82 area=57.9%
16:16:26 | INFO | scansmart.quality | Quality: grade=G00D sharpness=7000.4 contrast=69.9 skew=0.00 d
eg
16:16:27 | INFO | scansmart.pipeline | [2/6] Processing sample_02.jpg
16:16:27 | INFO | scansmart.detector | Detection: method=min_area_rect confidence=0.75 area=72.0%
16:16:27 | INFO | scansmart.quality | Quality: grade=G00D sharpness=6684.0 contrast=66.2 skew=0.00 d
eg
16:16:27 | INFO | scansmart.pipeline | [3/6] Processing sample_03.jpg
16:16:27 | INFO | scansmart.detector | Detection: method=contour confidence=0.89 area=60.0%
16:16:27 | INFO | scansmart.quality | Quality: grade=G00D sharpness=7004.1 contrast=68.7 skew=0.00 d
eg
16:16:27 | INFO | scansmart.pipeline | [4/6] Processing sample_04.jpg
16:16:27 | INFO | scansmart.detector | Detection: method=contour confidence=0.85 area=47.3%
16:16:27 | INFO | scansmart.quality | Quality: grade=G00D sharpness=7498.1 contrast=71.8 skew=0.00 d
eg
16:16:27 | INFO | scansmart.pipeline | [5/6] Processing sample_05.jpg
16:16:27 | INFO | scansmart.detector | Detection: method=contour confidence=0.78 area=51.1%
16:16:27 | INFO | scansmart.quality | Quality: grade=G00D sharpness=7392.3 contrast=71.3 skew=0.00 d
eg
16:16:28 | INFO | scansmart.pipeline | [6/6] Processing sample_06.jpg
16:16:28 | INFO | scansmart.detector | Detection: method=contour confidence=0.91 area=54.9%
16:16:28 | INFO | scansmart.quality | Quality: grade=G00D sharpness=7190.3 contrast=70.3 skew=0.00 d
eg
16:16:28 | INFO | scansmart.exporter | PDF written: outputs/scanned.pdf (6 page(s))
16:16:28 | INFO | scansmart.exporter | JSON report written: outputs/report.json
16:16:28 | INFO | scansmart.exporter | CSV report written: outputs/report.csv

FILE                STATUS  METHOD      CONF  GRADE  ms
-----
sample_01.jpg       ok      contour    0.82  G00D   162
sample_02.jpg       ok      min_area_rect 0.75  G00D   151
sample_03.jpg       ok      contour    0.89  G00D   106
sample_04.jpg       ok      contour    0.85  G00D   95
sample_05.jpg       ok      contour    0.78  G00D   115
sample_06.jpg       ok      contour    0.91  G00D   109
-----

Processed 6/6 (success 100%) | mean confidence 0.83 | mean time 123 ms
Grades: {'GOOD': 6, 'FAIR': 0, 'POOR': 0}

```

Screenshot 1 - Batch run of the CLI on the sample folder.

```

scansmart - accuracy evaluation

$ python tools/evaluate.py data/samples
16:16:28 | INFO | scansmart.detector | Detection: method=contour confidence=0.82 area=57.9%
16:16:28 | INFO | scansmart.detector | Detection: method=min_area_rect confidence=0.75 area=72.0%
16:16:28 | INFO | scansmart.detector | Detection: method=contour confidence=0.89 area=60.0%
16:16:28 | INFO | scansmart.detector | Detection: method=contour confidence=0.85 area=47.3%
16:16:28 | INFO | scansmart.detector | Detection: method=contour confidence=0.78 area=51.1%
16:16:28 | INFO | scansmart.detector | Detection: method=contour confidence=0.91 area=54.9%

FILE                METHOD      CONF  ERR %  ms  RESULT
-----
sample_01.jpg       contour    0.82   0.28   16   PASS
sample_02.jpg       min_area_rect 0.75   5.14   17   FAIL
sample_03.jpg       contour    0.89   0.23   14   PASS
sample_04.jpg       contour    0.85   0.32   13   PASS
sample_05.jpg       contour    0.78   0.30   13   PASS
sample_06.jpg       contour    0.91   0.28   13   PASS
-----

Accuracy          : 5/6 (83.3%)
Mean corner error: 1.09% of diagonal
Mean detect time : 14 ms

```

Screenshot 3 - Accuracy evaluation against synthetic ground truth.

```
scansmart - generated outputs

$ ls -lh outputs
total 19M
-rw-r--r-- 1 root root 761 Sep 17 16:16 report.csv
-rw-r--r-- 1 root root 3.6K Sep 17 16:16 report.json
-rw-r--r-- 1 root root 1.7M Sep 17 16:16 sample_01_comparison.png
-rw-r--r-- 1 root root 1.4M Sep 17 16:16 sample_01_detected.png
-rw-r--r-- 1 root root 23K Sep 17 16:16 sample_01_scanned.png
-rw-r--r-- 1 root root 1.7M Sep 17 16:16 sample_02_comparison.png
-rw-r--r-- 1 root root 1.4M Sep 17 16:16 sample_02_detected.png
-rw-r--r-- 1 root root 24K Sep 17 16:16 sample_02_scanned.png
-rw-r--r-- 1 root root 1.7M Sep 17 16:16 sample_03_comparison.png
-rw-r--r-- 1 root root 1.4M Sep 17 16:16 sample_03_detected.png
-rw-r--r-- 1 root root 23K Sep 17 16:16 sample_03_scanned.png
-rw-r--r-- 1 root root 1.6M Sep 17 16:16 sample_04_comparison.png
-rw-r--r-- 1 root root 1.4M Sep 17 16:16 sample_04_detected.png
-rw-r--r-- 1 root root 21K Sep 17 16:16 sample_04_scanned.png
-rw-r--r-- 1 root root 1.7M Sep 17 16:16 sample_05_comparison.png
-rw-r--r-- 1 root root 1.4M Sep 17 16:16 sample_05_detected.png
-rw-r--r-- 1 root root 24K Sep 17 16:16 sample_05_scanned.png
-rw-r--r-- 1 root root 1.7M Sep 17 16:16 sample_06_comparison.png
-rw-r--r-- 1 root root 1.4M Sep 17 16:16 sample_06_detected.png
-rw-r--r-- 1 root root 23K Sep 17 16:16 sample_06_scanned.png
-rw-r--r-- 1 root root 107K Sep 17 16:16 scanned.pdf
```

Screenshot 4 - Generated outputs: scanned pages, overlays, PDF and reports.

```
scansmart - version control

$ git log --oneline --graph --decorate
* 62fd2d2 (HEAD -> master) docs: generate project report PDF with wrapped tables and fitted cover
* 5b08d14 docs: README, problem statement, UML diagrams and result figures
* b0666b6 feat: CLI, batch pipeline, sample generator and accuracy evaluation
* 37e2601 feat: core CV package (config, preprocessing, detector, transform, enhancement) with tests
```

Screenshot 6 - Git commit history.

11. Testing Approach

Testing is organised in three levels: unit tests on individual operators, integration tests over the whole pipeline, and a quantitative evaluation against synthetic ground truth.

Test file	Tests	What it verifies
test_config.py	6	Defaults are valid; invalid mode, even kernels and inconsistent area ratios raise <code>InvalidConfigError</code> ; serialisation round-trip
test_preprocessing.py	7	Grayscale reduces channels and is idempotent; CLAHE preserves shape and dtype; denoising lowers the standard deviation of a noisy patch; auto-Canny returns a binary map; closing never removes edges
test_detector.py	7	Corner ordering is correct and idempotent; a synthetic page is located within 15 px of ground truth; the scale factor maps corners back correctly; a blank image falls back to the full frame, or raises when the fallback is disabled
test_transform.py	6	Output size equals the longest opposite edges and is clamped; warping a flat rectangle is the identity; a warped page becomes portrait and excludes the dark background; skew estimation detects a 5-degree rotation; tiny angles are ignored
test_enhancement_quality.py	11	scan mode is strictly binary; gray is single-channel; color keeps three channels; shadow removal flattens a gradient; unknown modes raise; sharpness falls after blurring; contrast of a flat image is zero; grading responds to low confidence
test_pipeline.py	12	Missing and corrupt files raise <code>ImageLoadError</code> ; folder discovery skips non-images; resizing preserves aspect ratio; save creates directories; end-to-end processing succeeds; a corrupt file is recorded as failed rather than raising; batch outputs and JSON/CSV reports are written

Total: **49 tests, all passing**. Tests build their own synthetic images in fixtures, so the suite needs no external data and runs in a few seconds.

```
$ pytest tests -v
...
49 passed in 4.1s

$ python tools/evaluate.py data/samples
Accuracy          : 5/6 (83.3%)
Mean corner error: 1.09% of diagonal
Mean detect time  : 14 ms
```

Validation testing

- **Input validation** - unsupported extensions are skipped, empty folders and missing paths raise `ImageLoadError`, corrupt bytes are detected by the decoder check.
- **Configuration validation** - even kernel sizes, unknown modes and impossible area ratios are rejected before any image is touched.
- **Output validation** - every processed page is measured and graded, so a silently wrong result (for example a full-frame fallback) is visible in the report as confidence 0.0 and grade POOR.

12. Challenges Faced

#	Challenge	How it was solved
1	Fixed Canny thresholds worked on one photo and failed on the next, depending on lighting.	Replaced them with median-based auto-Canny, so the thresholds adapt to each image.
2	The page border was detected as several disconnected edge fragments, so no closed contour existed.	Added morphological closing plus a dilation step to bridge the gaps before findContours.
3	A single approxPolyDP epsilon returned 5, 6 or 7 points instead of 4 for many contours.	Swept epsilon from 1 % to 8 % of the perimeter and accepted the first tolerance that produced a valid convex quadrilateral.
4	Warped pages occasionally came out mirrored or rotated by 90 degrees.	Traced it to inconsistent contour point order; fixed with the deterministic sum/difference corner ordering.
5	Text in the shadowed half of the page disappeared after global thresholding.	Introduced morphological background division followed by adaptive Gaussian thresholding.
6	Text blocks inside the page produced large contours that sometimes beat the page border.	Added the validity filter (12-99 % area, convexity) and an interior-angle score, so rectangle-like candidates win.
7	A page clipped by the frame edge (sample_02) has no closed boundary, so the contour strategy cannot succeed.	Accepted as a documented limitation: the minAreaRect fallback still returns a usable scan, but is reported with a lower confidence and a larger corner error (5.1 %).
8	Full-resolution detection was slow on large photographs.	Detection now runs on a 600 px copy and the corners are scaled back, cutting detection time to about 14 ms.
9	Evaluating accuracy needed labelled data that was not available.	Wrote a generator that renders pages through a known homography, producing photographs together with exact ground-truth corners.

13. Learnings & Key Takeaways

- **Pre-processing decides everything.** The detector improved far more from better filtering and adaptive thresholds than from any change to the contour search itself.
- **Parameters should be derived, not hard-coded.** Deriving Canny thresholds from the image median turned a brittle demo into something that works across lighting conditions.
- **Geometry is the core of the project.** Understanding that a photograph of a rectangle is a projective transform - and that a homography estimated from four point correspondences inverts it - is what makes the whole pipeline possible.
- **Degrade, do not fail.** A layered fallback with an honest confidence score is more useful than an algorithm that either succeeds perfectly or throws.
- **Measure, do not eyeball.** Synthetic ground truth converted vague impressions of quality into a single number (1.09 % mean corner error) that made regressions obvious.
- **Separation of concerns pays off in testing.** Because the CV modules are pure functions over arrays, 49 tests run in seconds without any test image files.
- **Classical CV is still competitive.** For a planar, high-contrast object, a well-tuned classical pipeline delivers sub-pixel-percentage accuracy in milliseconds on a CPU, with no training data at all.

14. Future Enhancements

Enhancement	Description
OCR integration	Feed the rectified page into Tesseract to produce a searchable PDF, the natural next stage for these outputs.
Multi-document detection	Detect and separate several receipts photographed together by keeping all valid quadrilaterals instead of the best one.
Page-curvature (dewarping) correction	Model book pages as a curved surface using text-line fitting, rather than assuming a plane.
Graphical / mobile interface	A drag-and-drop desktop GUI or an Android app with live camera preview and manual corner adjustment.
Learned detector as an extra strategy	Add a lightweight segmentation model as a fourth fallback for very low contrast scenes, keeping the classical path as the fast default.
Automatic border and staple removal	Detect and erase dark scan borders, punch holes and staple marks left after warping.
Parallel batch processing	Use multiprocessing to scan large folders across CPU cores.

15. References

- OpenCV Documentation - Image Filtering, Structural Analysis and Shape Descriptors, Geometric Image Transformations. <https://docs.opencv.org/>
- J. Canny, "A Computational Approach to Edge Detection", IEEE TPAMI, vol. 8, no. 6, 1986.
- D. Douglas and T. Peucker, "Algorithms for the reduction of the number of points required to represent a digitised line", Cartographica, 1973.
- R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004 (homography estimation, Chapter 4).
- R. Gonzalez and R. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018 (histogram equalisation, morphology, thresholding).

- S. Suzuki and K. Abe, "Topological structural analysis of digitised binary images by border following", CVGIP, 1985 (the algorithm behind cv2.findContours).
- N. Otsu, "A threshold selection method from gray-level histograms", IEEE Trans. SMC, 1979.
- K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization", Graphics Gems IV, 1994.
- J. Pech-Pacheco et al., "Diatom autofocusing in brightfield microscopy: a comparative study", ICPR 2000 (variance of the Laplacian focus measure).
- NumPy, reportlab and pytest official documentation.